SQL Week 2 Cheat Sheet (Data Analyst Focus)

Week 2 Learning Flow

Import CSV

↓

Understand Dataset

↓

String Functions

↓

Aggregate Functions

↓

CASE WHEN

↓

GROUP BY

↓

HAVING

↓

Aliases

↓

Date Functions

↓

Business Analysis

---

1. Import CSV File

Why?

Real companies provide data in:

- CSV
- Excel
- Reports

Before analysis, data must be imported.

Create Table

```
CREATE TABLE sales_data (
   order_id VARCHAR(20),
   order_date DATE,
   customer_name VARCHAR(100),
   country VARCHAR(50),
   state VARCHAR(50),
   city VARCHAR(50),
   region VARCHAR(50),
   sales DECIMAL(10,2),
   quantity INT,
   profit DECIMAL(10,2)
);
```

Import CSV

```
COPY sales_data
FROM 'C:/data/sales.csv'
DELIMITER ','
CSV HEADER;
```

---

2. String Functions

Purpose

Used for:

- Data Cleaning
- Text Formatting
- Standardization

UPPER()

Convert text to uppercase.

SELECT UPPER(customer_name)

FROM sales_data;

Output:

JOHN SMITH

LOWER()

Convert text to lowercase.

SELECT LOWER(city)

FROM sales_data;

LENGTH()

Count characters.

SELECT LENGTH(customer_name)

FROM sales_data;

TRIM()

Remove extra spaces.

SELECT TRIM(customer_name)

FROM sales_data;

CONCAT()

Join text.

SELECT CONCAT(customer_name,' - ',city)

FROM sales_data;

Output:

John Smith - Los Angeles

REPLACE()

Replace text.

SELECT REPLACE(city,'New York City','NYC')

FROM sales_data;

SUBSTRING()

Extract part of text.

SELECT SUBSTRING(customer_name FROM 1 FOR 4)

FROM sales_data;

Output:

John

---

3. Aggregate Functions

Purpose

Summarize data.

SUM()

Total value.

SELECT SUM(sales)

FROM sales_data;

AVG()

Average value.

SELECT AVG(profit)

FROM sales_data;

COUNT()

Count rows.

SELECT COUNT(*)

FROM sales_data;

MAX()

Highest value.

SELECT MAX(sales)

FROM sales_data;

MIN()

Lowest value.

SELECT MIN(profit)

FROM sales_data;

Real Analyst Questions

SELECT SUM(sales)

FROM sales_data;

Business Question:

What is total company revenue?

SELECT AVG(profit)

FROM sales_data;

Business Question:

What is average profit per order?

---

4. CASE WHEN

Purpose

Apply conditions and create categories.

Think:

IF → THEN

ELSE

Syntax

```
CASE
   WHEN condition THEN result
   ELSE result
END
```

Example

```
SELECT customer_name,
      sales,
      CASE
         WHEN sales > 1500 THEN 'High Sales'
         ELSE 'Low Sales'
      END AS sales_category
FROM sales_data;
```

Output

Sales	Category
1800	High Sales
900	Low Sales

Real Usage

- VIP Customers
- Profit/Loss Status
- Risk Categories
- Customer Segmentation

---

5. GROUP BY

Purpose

Group similar values and summarize them.

Syntax

```
SELECT column_name,
    AGG_FUNCTION(column)
FROM table
GROUP BY column_name;
```

Example

```
SELECT city,
    SUM(sales)
FROM sales_data
GROUP BY city;
```

Output

City	Total Sales
LA	5000
NYC	7000

Real Usage

- City-wise sales
- Region-wise profit
- Customer-wise spending

---

6. HAVING

Purpose

Filter grouped results.

Syntax

```
SELECT city,
    SUM(sales)
FROM sales_data
GROUP BY city
HAVING SUM(sales) > 5000;
```

Difference

WHERE

Filters rows.

```
WHERE sales > 1000
```

HAVING

Filters groups.

```
HAVING SUM(sales) > 5000
```

Easy Memory

WHERE → rows

HAVING → groups

---

7. Aliases

Purpose

Temporary name for columns/tables.

Column Alias

```
SELECT SUM(sales) AS total_sales
FROM sales_data;
```

Output:

total_sales

instead of sum

Table Alias

SELECT s.customer_name,

    s.sales

FROM sales_data AS s;

Real Usage

Makes reports cleaner and queries shorter.

---

8. Date Functions

Why Important?

Most business analysis is time-based.

Examples:

- Monthly Sales
- Yearly Revenue
- Daily Orders

CURRENT_DATE

Today's date.

SELECT CURRENT_DATE;

NOW()

Current date and time.

SELECT NOW();

EXTRACT()

Extract year/month/day.

Year

```
SELECT EXTRACT(YEAR FROM order_date)
FROM sales_data;
```

Month

```
SELECT EXTRACT(MONTH FROM order_date)
FROM sales_data;
```

Day

```
SELECT EXTRACT(DAY FROM order_date)
FROM sales_data;
```

Real Example

Monthly Revenue

```
SELECT EXTRACT(MONTH FROM order_date) AS month,
    SUM(sales)
FROM sales_data
GROUP BY month;
```

AGE()

Difference between dates.

```
SELECT AGE(CURRENT_DATE, order_date)
FROM sales_data;
```

DATE_TRUNC()

Group dates by month/year.

```
SELECT DATE_TRUNC('month', order_date),
    SUM(sales)
```

FROM sales_data

GROUP BY DATE_TRUNC('month', order_date);

---

9. SQL Execution Order

Many students memorize:

SELECT

FROM

WHERE

GROUP BY

HAVING

ORDER BY

LIMIT

This is writing order.

Actual Execution Order

1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY
7. LIMIT

---

10. Most Important Week 2 Business Queries

Total Revenue

SELECT SUM(sales)

FROM sales_data;

Average Profit

```
SELECT AVG(profit)
FROM sales_data;
```

Top 5 Highest Sales

```
SELECT *
FROM sales_data
ORDER BY sales DESC
LIMIT 5;
```

City-wise Sales

```
SELECT city,
    SUM(sales)
FROM sales_data
GROUP BY city;
```

Region-wise Profit

```
SELECT region,
    SUM(profit)
FROM sales_data
GROUP BY region;
```

High Sales Orders

```
SELECT order_id,
    sales,
    CASE
        WHEN sales > 1500 THEN 'High'
        ELSE 'Low'
    END AS sales_level
FROM sales_data;
```

Week 2 Revision in 30 Seconds

Import Data

↓

Clean Text (String Functions)

↓

Summarize Data (Aggregate Functions)

↓

Create Categories (CASE WHEN)

↓

Group Data (GROUP BY)

↓

Filter Groups (HAVING)

↓

Rename Output (Aliases)

↓

Analyze Time Trends (Date Functions)

↓

Generate Business Insights

Golden Rule for Data Analysts

Don't ask:

Which SQL function should I use?

Ask:

What business question am I trying to answer?

Then choose the SQL function that helps answer it. This is how real analysts think.